

20 Questions SQL

Test technique /20

Avant ton entretien Data Analyst

Durée conseillée : 20 min · 3 niveaux · corrigé détaillé

Objectif : mesurer tes réflexes SQL réels, pas seulement ta syntaxe. Ce test vérifie si tu sais quand un JOIN duplique tes lignes, quel grain tu lis avant d'agréger et pourquoi un total peut devenir faux.

Repères : 12/20 = junior, 17/20 = profil qu'on garde.

Candidat	_____	Date	_____	Note	_____ /20
----------	-------	------	-------	------	-----------

■ ■ ■ Niveau 1 : Fondamentaux SQL

Question 1 : Que renvoie la requête `SELECT DISTINCT ville FROM clients` ?

- a) La liste des villes sans doublon.
- b) Toutes les lignes de la colonne `ville`, doublons compris.
- c) Le nombre de villes distinctes.
- d) Uniquement les villes qui apparaissent en double.

Question 2 : Quelle clause filtre les lignes selon une condition, avant tout regroupement ?

- a) `HAVING`
- b) `GROUP BY`
- c) `WHERE`
- d) `ORDER BY`

Question 3 : Un `INNER JOIN` entre `commandes` et `clients` retourne :

- a) Toutes les commandes, même celles sans client correspondant.
- b) Uniquement les lignes ayant une correspondance dans les deux tables.
- c) Tous les clients, même ceux sans commande.
- d) Le produit cartésien des deux tables.

Question 4 : Un `LEFT JOIN` de `clients` vers `commandes` garantit que le résultat contient :

- a) Seulement les clients ayant au moins une commande.
- b) Seulement les correspondances exactes entre les deux tables.
- c) Toutes les commandes, même celles sans client rattaché.
- d) Tous les clients, même ceux sans commande (colonnes de `commandes` à `NULL`).

Question 5 : La requête `SELECT ville, COUNT(*) FROM clients GROUP BY ville` renvoie :

- a) Le nombre total de clients, toutes villes confondues.
- b) Une ligne par client.
- c) Une ligne par ville, avec son nombre de clients.
- d) Une erreur, car `COUNT(*)` est interdit avec `GROUP BY`.

Question 6 : Sans préciser `ASC` ni `DESC`, `ORDER BY date_commande` trie :

- a) Du plus récent au plus ancien (décroissant).
- b) Du plus petit au plus grand (croissant, `ASC` par défaut).
- c) Dans l'ordre d'insertion des lignes.
- d) De façon non déterministe.

■ ■ ■ Niveau 2 : Pièges classiques

Question 7 : Un client a 3 commandes. Un `LEFT JOIN` de `clients` vers `commandes` produit, pour ce client :

- a) 1 seule ligne, les commandes étant agrégées automatiquement.
- b) 3 lignes : le client est répété pour chaque commande.
- c) 0 ligne.
- d) Une erreur de cardinalité.

Question 8 : Sur un groupe où tous les `montant` valent `NULL`, `SUM(montant)` retourne :

- a) `NULL`.
- b) 0.
- c) Une erreur.
- d) La valeur du dernier `montant` non nul.

Question 9 : Sur une colonne `email` contenant des `NULL`, `COUNT(email)` renvoie :

- a) Le même résultat que `COUNT(*)`.
- b) Toujours 0.
- c) Le nombre de lignes où `email` n'est pas `NULL`.
- d) Le nombre de valeurs distinctes d'`email`.

Question 10 : Pour compter le nombre de clients uniques ayant passé commande, on écrit :

- a) COUNT(*)
- b) COUNT(client_id)
- c) DISTINCT COUNT(client_id)
- d) COUNT(DISTINCT client_id)

Question 11 : La requête `SELECT prix * quantite AS total FROM ventes WHERE total > 100` :

- a) Fonctionne, l'alias `total` est reconnu dans le `WHERE`.
- b) Échoue, l'alias du `SELECT` n'existe pas encore au moment du `WHERE`.
- c) Fonctionne mais ignore silencieusement le filtre.
- d) Ne fonctionne qu'en PostgreSQL.

Question 12 : Dans `WHERE client_id NOT IN (SELECT id_referent FROM blacklist)`, si la sous-requête contient au moins un `NULL`, la requête renvoie :

- a) Le résultat attendu, le `NULL` étant ignoré.
- b) Une erreur de syntaxe.
- c) Toutes les lignes de la table.
- d) Aucune ligne, car la condition devient « inconnue » pour chaque ligne.

Question 13 : Pour ne garder que les villes ayant plus de 10 clients (`COUNT(*) > 10`), on filtre avec :

- a) `WHERE COUNT(*) > 10`
- b) `HAVING COUNT(*) > 10`
- c) `ORDER BY COUNT(*) > 10`
- d) `GROUP BY COUNT(*) > 10`

■■■ Niveau 3 : Raisonnement sous contrainte

Question 14 : Deux lignes sont à égalité au 1er rang. Quelle fonction attribue les rangs 1, 1, puis 3 (en sautant le 2) ?

- a) `ROW_NUMBER()`
- b) `DENSE_RANK()`
- c) `RANK()`
- d) `NTILE()`

Question 15 : Pour ne garder qu'une seule ligne par client (la plus récente), on numérote avec :

- a) `ROW_NUMBER() OVER (PARTITION BY client_id ORDER BY date DESC)`.
- b) `RANK()`, qui peut renvoyer plusieurs lignes en cas d'égalité.
- c) `COUNT(*) OVER (PARTITION BY client_id)`.
- d) `DENSE_RANK()` sans `PARTITION BY`.

Question 16 : On veut garder uniquement les lignes où `ROW_NUMBER() = 1`. Pourquoi ne peut-on pas l'écrire dans le `WHERE`?

- a) Parce que `ROW_NUMBER` n'existe pas en SQL standard.
- b) Parce que le `WHERE` n'accepte pas les comparaisons d'entiers.
- c) Parce qu'il faut d'abord un `GROUP BY`.
- d) Parce que la fonction de fenêtre est calculée après le `WHERE` (il faut une CTE, une sous-requête ou `QUALIFY`).

Question 17 : Quel est l'ordre logique d'exécution correct d'une requête?

- a) `SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY`.
- b) `FROM, SELECT, WHERE, GROUP BY, HAVING, ORDER BY`.
- c) `FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY`.
- d) `WHERE, FROM, GROUP BY, SELECT, HAVING, ORDER BY`.

Question 18 : Pour trouver les clients sans aucune commande (anti-join), l'approche la plus fiable est :

- a) Un `LEFT JOIN` sur commandes, puis `WHERE commandes.client_id IS NULL`.
- b) Un `INNER JOIN` sur commandes, puis `COUNT(*) = 0`.
- c) Un `NOT IN` sur une sous-requête pouvant contenir des `NULL`.
- d) Un simple `GROUP BY client_id`.

Question 19 : Après un `JOIN` qui duplique des lignes, un `SUM(montant)` devient faux. La cause :

- a) `SUM` ne supporte pas les requêtes avec `JOIN`.
- b) Les `NULL` faussent toujours la somme.
- c) Il manque un `ORDER BY` avant l'agrégation.
- d) Le grain a changé : chaque `montant` est compté plusieurs fois.

Question 20 : Une moyenne mobile « 7 jours » écrite avec `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` est fausse si :

- a) Certains jours manquent : on moyenne alors 7 lignes, pas 7 jours.
- b) La table est triée par date.
- c) On utilise `AVG` au lieu de `SUM` divisé par 7.
- d) Le partitionnement par produit est défini.

■■ Solutions et Explications

- **1 : a)** `DISTINCT` retire les doublons et renvoie la liste des villes uniques, sans les compter.
- **2 : c)** `WHERE` filtre ligne par ligne avant tout regroupement; aucune fonction d'agrégation n'est admise à ce stade.
- **3 : b)** Un `INNER JOIN` ne conserve que les lignes appariées des deux côtés; les orphelines disparaissent.
- **4 : d)** Le `LEFT JOIN` garde toutes les lignes de gauche; les colonnes de droite passent à `NULL` faute de correspondance.
- **5 : c)** `GROUP BY ville` replie les lignes par ville et `COUNT(*)` compte les clients de chaque groupe.
- **6 : b)** Sans précision, `ORDER BY` trie en ordre croissant : `ASC` est la valeur par défaut.
- **7 : b)** Le `LEFT JOIN` suit le grain de la table de droite : 3 commandes dupliquent le client sur 3 lignes.
- **8 : a)** `SUM` sur un groupe entièrement `NULL` renvoie `NULL`, pas 0; il faut `COALESCE` pour forcer un zéro.
- **9 : c)** `COUNT(col)` ignore les `NULL` et ne compte que les valeurs présentes, contrairement à `COUNT(*)`.
- **10 : d)** `COUNT(DISTINCT client_id)` dédoubleonne avant de compter; `COUNT(client_id)` compterait chaque commande.
- **11 : b)** L'alias du `SELECT` est résolu après le `WHERE` : il faut répéter l'expression ou passer par une sous-requête ou CTE.
- **12 : d)** Avec un `NULL` dans la liste, `NOT IN` renvoie « inconnu » pour chaque ligne et ne sort rien; préfère `NOT EXISTS`.
- **13 : b)** Un filtre sur un agrégat se met dans `HAVING`, appliqué après le `GROUP BY`; `WHERE` agirait avant l'agrégation.
- **14 : c)** `RANK` laisse un trou après une égalité (1, 1, 3); `DENSE_RANK` ferait 1, 1, 2 et `ROW_NUMBER` 1, 2, 3.
- **15 : a)** `ROW_NUMBER` avec `PARTITION BY` garantit une seule ligne par client; `RANK` pourrait en renvoyer plusieurs en cas d'égalité.
- **16 : d)** Une fonction de fenêtre est évaluée après le `WHERE`; pour la filtrer, passe par une CTE, une sous-requête ou `QUALIFY`.
- **17 : c)** L'ordre logique réel : `FROM`, puis `WHERE`, `GROUP BY`, `HAVING`, `SELECT` (et les fonctions de fenêtre), enfin `ORDER BY` et `LIMIT`.
- **18 : a)** Le `LEFT JOIN` suivi de `IS NULL` côté droit isole les clients sans commande, sans le piège des `NULL` de `NOT IN`.
- **19 : d)** La duplication change le grain : chaque `montant` est répété et compté plusieurs fois, d'où un total gonflé. Agrège avant de joindre.
- **20 : a)** `ROWS` compte des lignes physiques : si des jours manquent, 6 lignes en arrière ne font pas 6 jours. Pour de vrais jours, utilise `RANGE` avec un intervalle de dates.